Project Development Phase
Model Performance Test

Date	13 October 2026
Team ID	6ab4a5100fc666a751b4361b
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	

Model Performance Testing:

Projectteamshallfillthefollowinginformationinmodel performance testing template.

S.No.	Parameter	Values	Screenshot
1.	Model Summary	**Project:** Implement Client Script & UI Policy (Incident) **Problem Statement** Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behavior and validation directly at the user interface level. **Objective** The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met. The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform.	
2.	UI Policy Configuration	**Task 1: Create UI Policy on Incident** Name: High Impact Control Table: Incident Active: true Condition: Impact is 1 – High UI Policy Action: Assignment group — Check mandatory Reverse if false: true **Task 2: Create UI Policy Action – Urgency** Field name: Urgency Read-only: true Visible: Leave this field as is (do not change).	

S.No.	Parameter	Values	Screenshot
3.	Client Script Configuration	**Task 3: Create onChange Client Script** Name: Auto set urgency for high impact Table: Incident Type: onChange Field name: Impact Active: true When Impact is High, Urgency is set to High and an information message is displayed. **Task 4: Create onSubmit Client Script (Save Validation)** Name: Prevent save if Assigned To missing Table: Incident Type: onSubmit Active: true When Impact is High and Assigned To is empty, saving is prevented and an error is shown. **Task 5: Create onCellEdit Client Script** Name: Prevent state change via list edit Table: Incident Type: onCellEdit Field name: State Active: true Displays an alert and blocks direct State changes through list editing.	
4.	Testing Scenarios & Results	**Activity 1: Test Mandatory Enforcement** — Set Impact to High, leave Assigned To empty, and submit. The Incident should not be saved and an error message should appear. **Activity 2: Test Successful Save** — Fill Assigned To and submit. The Incident should save successfully, with urgency auto-setting, read-only behavior, and State restrictions working as expected. **Activity 3: Reverse Condition Test** — Change Impact from High to Medium. Assigned To should no longer be mandatory, Urgency should be editable, and the record should save successfully. **Activity 4: Test List Edit Blocking** — Try editing State directly from the Incident list. An alert should appear and the State value should remain unchanged. **Activity 5: Test Form-Based Update** — Change State on the Incident form and update. The State change should save successfully and other configured rules should remain in effect.	
5.	Skills	Incident Management UI Policy UI Policy Actions Client Scripts Form Validation	

S.No.	Parameter	Values	Screenshot
6.	Conclusion	The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields like Assigned To and Urgency respond appropriately to changes in Impact, maintaining clean and consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.	